

Data Warehouse Solution for Online Furniture Retailer

(Online Möbelhandel)

End-to-End Implementation of an Operational Business DB, Dimensional Star Schema, and ETL Pipeline

1. MOTIVATION & BACKGROUND

A German online furniture retailer (comparable to home24) sells furniture and home accessories to private customers throughout Germany. The operational shop database is designed exclusively for processing individual transactional orders answering "what is in order 41725?" quickly. However, it cannot efficiently answer high-level analytical queries such as "how did revenue per product category develop over the last two years?" without executing expensive joins and aggregations across the live database.

This project builds a compact yet complete data warehouse solution for the retailer: a normalized operational source database, a dimensional target model, and a Python-based ETL process connecting the two systems.

2. PROJECT OBJECTIVES

1. Operational Source (Business DB)

Design and implement a relational database in 3NF representing the shop's order-taking process.

2. Target Data Warehouse

Design a Star Schema dimensional model and implement an automated ETL pipeline in Python to transform and load transactional data.

3. Slowly Changing Dimensions (SCD)

Analyze dimension attribute behavior and demonstrate precise handling of SCD Types 0, 1, and 2.

4. Business Intelligence & Analytics

Formulate and execute analytical SQL queries against the warehouse to deliver key operational insights.

3. BUSINESS DB DESIGN

3.1 ERM DESIGN

- The primary domain covers the order-taking process: a customer (entity *Customer*) places an order (entity *OrderHeader*) containing one or more order lines (entity *OrderLine*).
- Each order line is associated with the sale of a specific product (entity *Product*) in a certain quantity. Each product might be discounted for a specific order (e.g., due to a promotional campaign).
- Each product is assigned to a category (e.g., Sofa, Tisch etc.). A category might be associated with a parent category (e.g., Sofa is associated with the parent category *Wohnzimmer*).
- Each product is supplied by a specific supplier (entity *Supplier*).

The corresponding ERM diagram is shown below:

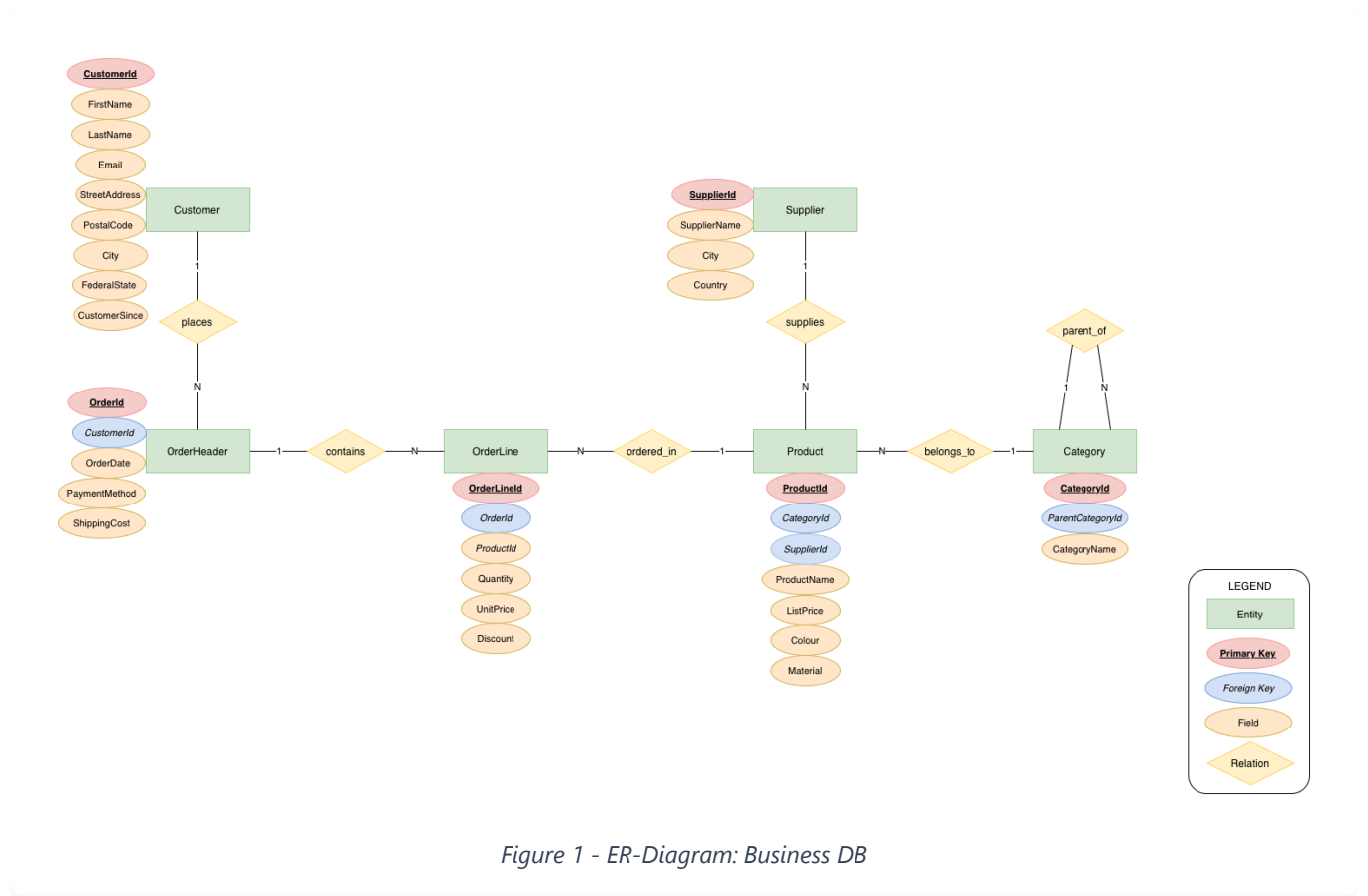


Figure 1 - ER-Diagram: Business DB

3.2 RELATIONAL SCHEMA (with Data Types)

```

Supplier(
  SupplierId    INTEGER    PK  AUTOINCREMENT,
  SupplierName  TEXT       NOT NULL  UNIQUE,
  City          TEXT       NOT NULL,
  Country       TEXT       NOT NULL  DEFAULT 'Germany'
)

Category(
  CategoryId    INTEGER    PK  AUTOINCREMENT,
  CategoryName  TEXT       NOT NULL  UNIQUE,
  ParentCategoryId INTEGER  FK -> Category(CategoryId)  NULL
)

Product(
  ProductId     INTEGER    PK  AUTOINCREMENT,
  ProductName   TEXT       NOT NULL  UNIQUE,
  ListPrice     REAL       NOT NULL  CHECK(>0),
  Colour        TEXT,
  Material      TEXT,
  CategoryId    INTEGER    NOT NULL  FK -> Category(CategoryId),
  SupplierId    INTEGER    NOT NULL  FK -> Supplier(SupplierId)
)

Customer(
  CustomerId    INTEGER    PK  AUTOINCREMENT,
  FirstName     TEXT       NOT NULL,

```

```

    LastName      TEXT      NOT NULL,
    Email          TEXT      NOT NULL  UNIQUE,
    StreetAddress  TEXT      NOT NULL,
    PostalCode     TEXT      NOT NULL,
    City           TEXT      NOT NULL,
    FederalState   TEXT      NOT NULL,
    CustomerSince  TEXT      NOT NULL  -- ISO date YYYY-MM-DD
)

OrderHeader(
    OrderId        INTEGER    PK  AUTOINCREMENT,
    OrderDate       TEXT      NOT NULL,  -- ISO date YYYY-MM-DD
    PaymentMethod   TEXT      NOT NULL  CHECK IN ('credit_card', 'paypal', 'bank_transfer', 'invoice'),
    ShippingCost    REAL      NOT NULL  CHECK(>=0),
    CustomerId      INTEGER    NOT NULL  FK -> Customer(CustomerId)
)

OrderLine(
    OrderLineId    INTEGER    PK  AUTOINCREMENT,
    Quantity        INTEGER    NOT NULL  CHECK(>0),
    UnitPrice       REAL      NOT NULL  CHECK(>0),
    Discount        REAL      NOT NULL  CHECK(>=0 AND <1),
    OrderId         INTEGER    NOT NULL  FK -> OrderHeader(OrderId),
    ProductId       INTEGER    NOT NULL  FK -> Product(ProductId)
)

```

3.3 NORMAL FORM VERIFICATION

>

Form	Check	Result
1NF	Atomic attributes, no repeating groups, single-column PKs	✓
2NF	No partial dependencies (all PKs are surrogates - no composite PK)	✓
3NF	<i>PostalCode</i> is not derived from <i>City</i> . <i>City</i> is stored independently. <i>UnitPrice</i> != <i>ListPrice</i> (dynamic pricing is considered). Category attributes separated.	✓

3.4 SYNTHETIC DATA GENERATION

- Suitable data for a suitable business DB was not available from real sources. Business data was generated synthetically.
- The python *faker* module was used for generating realistic customer names and addresses. Because faker generates PLZ and cities independently, faker generated PLZ/city associations are not correct. Therefore, a separate script was used to generate correct PLZ/city associations for 75 cities (city list suggested by AI). This was necessary because analytical questions can be modelled on PLZ/City association.
- Faker generated street names do match cities - but it is not important because no street level analytics are considered.
- Product, category and supplier name tables were generated using AI suggestions.

The data is generated and imported into the SQLite3 database using a python script. Summary of the generated data is presented in the table below:

Entity	Count	Method
Supplier	20	Hardcoded (10 Germany, 10 neighboring countries)
Category	15	5 top-level + 10 sub-categories (hardcoded tree)
Product	120	Name generated using Templates (e.g., Sofa) + adjectives (e.g., Classic); price 50€–2,500€ random
Customer	5,000	Faker de_DE ; Email derived from name; PLZ/City generated using a simple algorithm for 75 cities
OrderHeader	30,000	Random dates 2024-01-01 to 2025-12-31; weighted payment methods
OrderLine	~85,500	1-5 lines/order; UnitPrice = ListPrice $\pm$ 10%; 80% zero Discount (random generation)

3.5 STORAGE ANALYSIS

Storage analysis was carried out for SQLite3 database. Following assumptions were made (some of the values were suggested by AI - proper SQ Lite documentation was not found):

- 8 bytes per INTEGER column
- 8 bytes overhead per row (This was suggested by AI - could not find a value in SQLite documentation)
- For customer name, e-mail and address fields the average length (in bytes) was taken for one data generation instance. With each new run of the data generation process, these average lengths may vary slightly (and the DB storage will also vary).

Table	Column	Type	Bytes/row
Supplier	SupplierId	INTEGER	8
	SupplierName	TEXT	~22
	City	TEXT	~14
	Country	TEXT	~12
	Row overhead		8
Supplier total		~64 B/row	20 rows = 1.3 KB
Category	CategoryId	INTEGER	8
	CategoryName	TEXT	~17
	ParentCategoryId	INTEGER (nullable)	8
	Row overhead		8
Category total		~41 B/row	15 rows = 0.6 KB
Product	ProductId	INTEGER	8

Table	Column	Type	Bytes/row
	ProductName	TEXT	~22
	ListPrice	REAL	8
	Colour	TEXT	~10
	Material	TEXT	~12
	CategoryId	INTEGER	8
	SupplierId	INTEGER	8
	Row overhead		8
Product total		~84 B/row	120 rows = 10 KB
Customer	CustomerId	INTEGER	8
	FirstName	TEXT	~10
	LastName	TEXT	~12
	Email	TEXT	~27
	StreetAddress	TEXT	~32
	PostalCode	TEXT	7
	City	TEXT	~14
	FederalState	TEXT	~17
	CustomerSince	TEXT	12
	Row overhead		8
Customer total		~147 B/row	5,000 rows = 720 KB
OrderHeader	OrderId	INTEGER	8
	OrderDate	TEXT	12
	PaymentMethod	TEXT	~14
	ShippingCost	REAL	8
	CustomerId	INTEGER	8
	Row overhead		8
OrderHeader total		~58 B/row	30,000 rows = 1.7 MB

Table	Column	Type	Bytes/row
OrderLine	OrderLineId	INTEGER	8
	Quantity	INTEGER	8
	UnitPrice	REAL	8
	Discount	REAL	8
	OrderId	INTEGER	8
	ProductId	INTEGER	8
	Row overhead		8
OrderLine total		~56 B/row	85,500 rows = 4.7 MB

Table	Rows	Avg row (bytes)	Estimated
Supplier	20	64	1.3 KB
Category	15	41	0.6 KB
Product	120	84	10 KB
Customer	5,000	147	720 KB
OrderHeader	30,000	58	1.7 MB
OrderLine	85,500	56	4.7 MB
Subtotal			~7.1 MB

4. CORE ANALYTICAL QUESTIONS

The dimensional warehouse is specifically designed to evaluate two core business queries alongside supplemental extensions:

Question	Business Description	Dimensions Required
Q1 Revenue and Discount by category over time	How does net revenue as well as Discounts develop per top-level product category and Quarter?	DIM_Date, DIM_Product
Q2 Top 10 PLZ zones by net revenue	Which 10 postal code zones (first 2 digits of PLZ) generate the maximum net revenue?	DIM_Customer
Q3 Top 2 product categories per federal state by net revenue	What are the two highest net revenue generating product categories per federal state?	DIM_Customer, DIM_Product
Q4 Peak order volume by weekday	Which weekdays see the highest order volume across the two-year period?	DIM_Date
Q5 Peak calendar weeks (Top 10)	Which calendar weeks see the highest order volume?	DIM_Date
Q6 Revenue by price segment	How does revenue compare across Budget / Mid-range / Premium product segments?	DIM_Product
Q7 Federal state revenue growth: Year 1 (2024) vs Year 2 (2025)	Which federal states show the strongest revenue growth from 2024 to 2025?	DIM_Customer, DIM_Date
Q8 Top 3 suppliers per quarter	Which 3 suppliers generate the most net revenue per Quarter?	DIM_Supplier, DIM_Date
Q9 Supplier Discount behaviour vs. order volume	Which suppliers' products carry the highest average Discount rate, and does that correlate with order volume?	DIM_Supplier

5. TARGET WAREHOUSE SCHEMA

A star schema layout where the grain of the fact table is **one row per order line**.

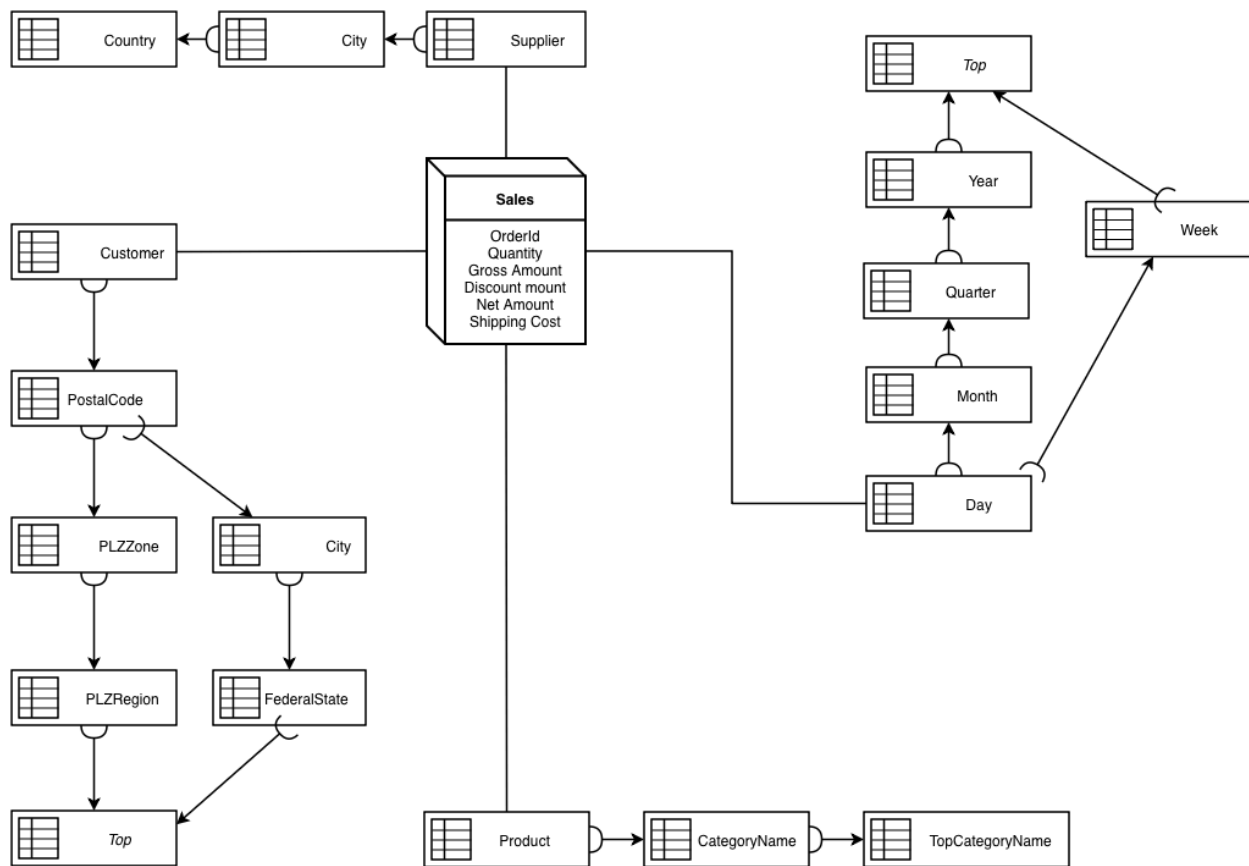


Figure 2 - mER-Diagram: Data Warehouse

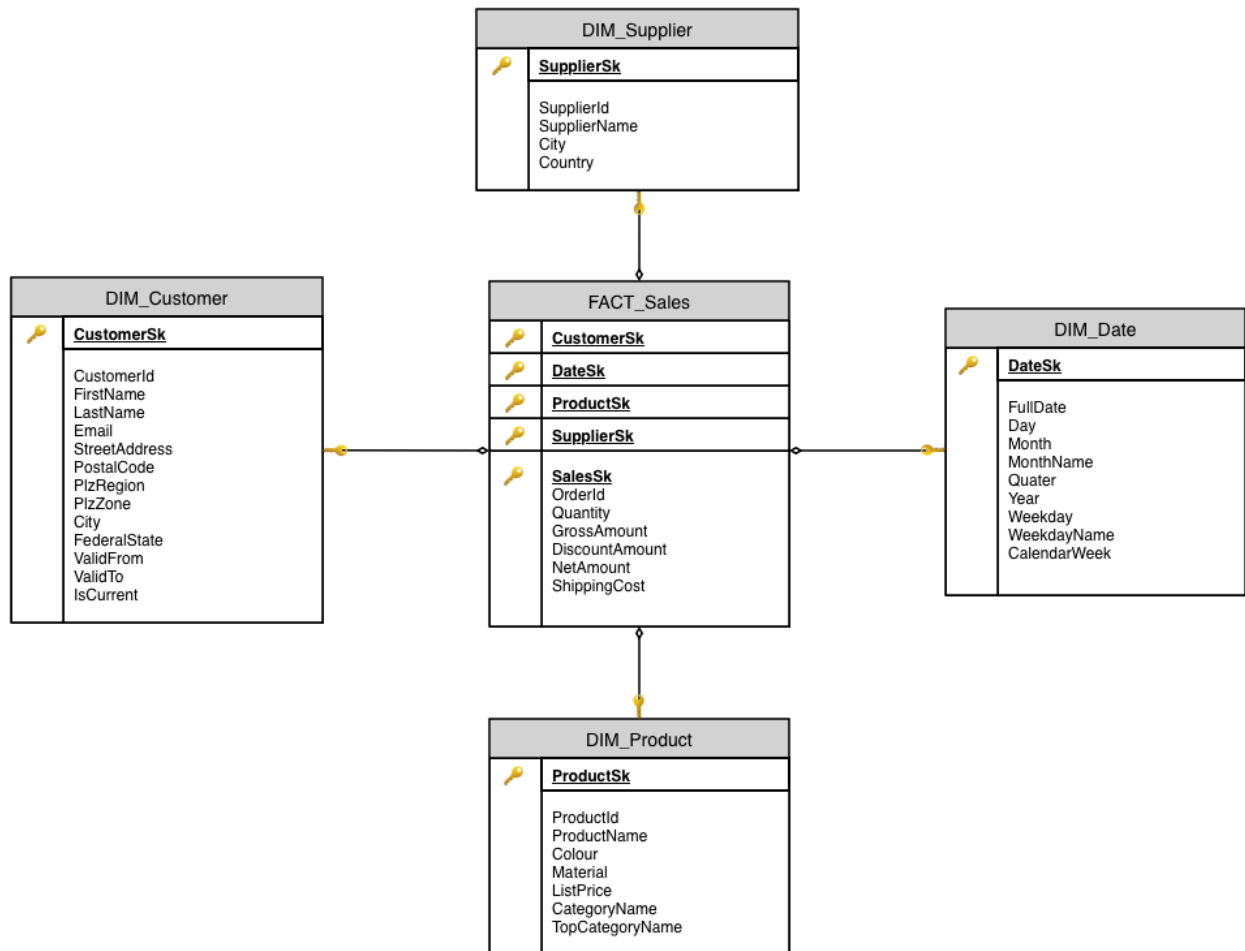


Figure 3 - Star Schema: Data Warehouse

6. ETL PROCESS & TRANSFORMATION LOGIC

6.1 BRIEF CODE OVERVIEW

- Implemented using python 3.11+, and SQLite3
- The code is organized in two directories - `sql/` and `python/`
- `sql/` contains all SQL scripts for creating and transforming tables in the data warehouse. These scripts are executed from python code during the data generation and etl process.
 - The scripts for RAW_ to FULL_ table transformations are included in `sql/transform/`.
 - The scripts for analytics questions are included in `sql/analytics/`.
- `python/` contains Python scripts for orchestrating the ETL process.
 - `python/data_gen/` contains scripts to generate and import business data
 - `python/etl/` contains scripts to orchestrate the ETL process (See details below)
 - `python/run.py` is a menu driven orchestrator script. The script offers menu choices to generate the business DB, run the ETL process, and execute analytics queries. Additionally, it provides an option to generate new order and customer data so as to test the SCD implementation.

6.2 ETL PROCESS

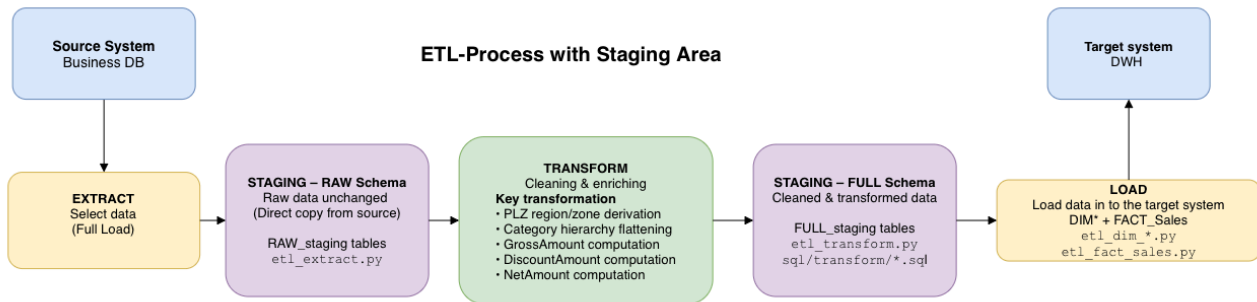


Figure 4 - ETL Process

- *run_all_etl* (entire data pipeline)
 - *Stage 1: run_etl_extract()* Copies business DB to RAW_ tables in the data warehouse
 - *apply_dwh_ddl* Creates RAW_ , FULL_ , DIM_ , FACT_ tables if they do not exist (`sql/02_create_dwh.sql`)
 - *extract_to_raw*
 - Iterates over _RAW_MAP (Business DB to RAW_ table name mapping, e.g. Customer → RAW_BusinessDB_Customer)
 - For each RAW_ table, deletes data and copies from the corresponding Business DB table.
 - *Stage 2: run_etl_transform()* → Transforms RAW_ tables into FULL_ tables
 - Executes the 3 transformation scripts for the 3 FULL_ tables (from `sql/transform/`)
 - Each script performs the following: Deletes existing table data and copies data from RAW_ tables while transforming it.
 - For customers → Additional *PlzZone* and *PlzRegion* are added
 - For products → Category name and top category names are added by joining with the Category table (flattening of category hierarchy)
 - For sales → Full sales table is created by joining order header, order line, and product data (i.e. each line corresponds to an order line)
 - *Stage 3: Update dimension and fact tables*
 - *run_etl_dim_date()*
 - DIM_Date stores all unique order dates with a surrogate key in the form <<YYYYMMDD>> , i.e. by replacing the - in the original date with an empty string.
 - *run_etl_dim_supplier()*
 - Inserts new supplier IDs with a surrogate key into DIM_Supplier if new suppliers are added to the business DB.
 - *run_etl_dim_product()*
 - Inserts a new product ID with a surrogate key into DIM_Product if new products are added to the business DB.
 - If any attribute of a product is updated in the business DB, the corresponding record in DIM_Product is also updated (SCD 1 approach).
 - *run_etl_dim_customer()*
 - Inserts a new customer ID with a surrogate key into DIM_Customer if new customers are added to the business DB.

- If the address of a customer is changed, the corresponding record in `DIM_Customer` is updated by changing `IsCurrent` to 0 and inserting a new record with the updated address (`IsCurrent = 1`) (SCD 2 approach). See the SCD logic table below for details.
- `run_etl_fact_sales()`
 - Inserts new sales records into `FACT_Sales` if new sales are added to the business DB.
 - For customers, the current surrogate key from `DIM_Customer` is used to maintain the relationship between sales and customers.

7. SCD LOGIC

Dimension	SCD Type	Changed attributes	Justification
DIM_Date	0	-	Dates immutable
DIM_Supplier	0	-	Supplier identity stable
DIM_Product	1 (overwrite)	ProductName, ListPrice, Colour, Material	Corrections tolerated; no historical query requires old values
DIM_Customer	2 (expire + insert)	PostalCode, StreetAddress, City, FederalState, PlzRegion, PlzZone	Relocation may change PLZ and street address. Historical order assignment matters for analytics.

The following attributes are used to track changes in customer address:

Column	Type	Description
ValidFrom	DATE	Row start date (CustomerSince for first load; ETL date on change)
ValidTo	DATE	Row end date (NULL = active)
IsCurrent	INTEGER	1 = active; 0 = historical